

Problema 1.4

Dato il linguaggio $L = \{w \in \{0,1\}^* \mid |w|_0 = |w|_1\}$, dimostrare che $L \notin \text{REG}$

SUPPONIAMO PER ASSURDO CHE L SIA REGOLARE ALLORA DEVE VALERE IL PUMPING LEMMA PER I LINGUAGGI REGOLARI

CONSIDERIAMO $w = 0^p 1^p$ PERCHÉ $|w| \geq p$ E DIVIDIAMO w IN 3 PARTI xyz $|xy| \leq p$ per ipotesi quindi

$$w = 0^m 0^{p-m} 1^p$$

$$xy = 0^m \text{ e } z = 0^{p-m} 1^p$$

può essere ripetuto con $k \geq 0, k \in \mathbb{N}$ $xy^k z \in L$.

$$xy = 0^{m-h} 0^h \text{ quindi } xyz = 0^{m-h} 0 0^{p-m} 1^p$$

con $k=0$

$$xyz = 0^{m-h} (0^h)^{k=0} 0^{p-m} 1^p = 0^{m-h} 0^{p-m} 1^p = 0^{p-h} 1^p$$

con $k=0$ $xy^0 z \notin L$ CONTRADDICENDO LA CONDIZIONE DEL PUMPING LEMMA ... ECC...



Problema 1.5

Dato il linguaggio $L = \{1^{n^2} \mid n \in \mathbb{N}\}$, dimostrare che $L \notin \text{REG}$

DIAMO PER VERO CHE $L \in \text{REG}$ QUINDI, PRESA $w \in L \mid |w| \geq p$ DOVE p È LA LUNGHEZZA DEL PUMPING.

$w = 1^{p^2}$, POSSIAMO DIVIDERE QUESTA STRINGA IN 3 PARTI xyz dove $|xy| \leq p$

$$w = 1^p 1^p = 1^{p-m} 1^m 1^p \text{ dove ...}$$

$$xy = 1^{p-m}$$

$$z = 1^m 1^p \text{ supponiamo per il pumping lemma che } \forall k \in \mathbb{N}$$

$xy^k z \in L$ e che $|y| > 0$ se il linguaggio L è REGOLARE

$$w = 1^{p-m-h} 1^h 1^m 1^p, \text{ se } k=0$$

$w_2 = 1^{p-m-h} (1^h)^0 1^m 1^p = 1^{p-h} 1^p$ e $w_2 \notin L$ NEGANDO QUINDI la condizione espone prova del pumping lemma, $L \notin REG$

Problema 1.7

Dato il linguaggio $L = \{c^4 a^n b^m \mid n, m \in \mathbb{N}, n \geq m\}$, dimostrare che $L \notin REG$

SUPPONIAMO CHE $L \in REG$ quindi deve rispettare il pumping lemma, prendiamo $w : |w| \geq p$ dove p è la lunghezza del pumping $w = c^4 a^{p-x-4} a^x b^p$

dividiamo w in 3 parti xgz

$$xy = c^4 a^{p-x-4}$$

$$z = a^x b^p$$

$$\text{DATO CHE } |xy| \leq p$$

$$\text{ma, dato che } |y| > 0$$

$$w = c^4 a^{p-x-4-x} a^x a^x b^p$$

$$\text{DOVE } a^x = y$$

$$c^4 a^{p-x-4-x} a^x = xy$$

$$a^x b^p = z$$

$$\text{PER } k \in \mathbb{N}, k=0$$

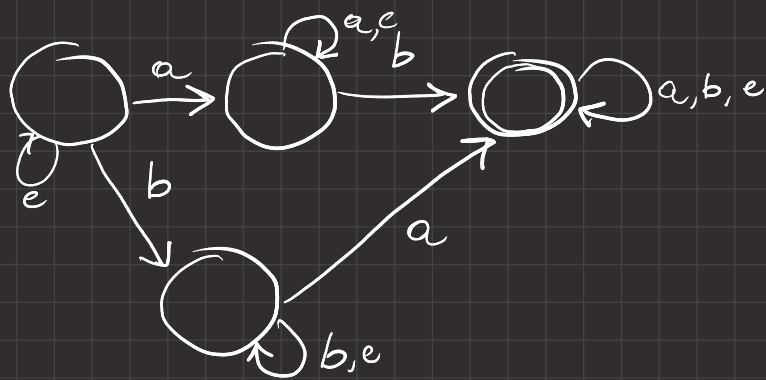
$w_2 = xy^0z = c^4 a^{p-4} b^p \notin L \Rightarrow L \notin REG$ perché per essere regolare, ogni stringa $w : |w| \geq p$, $w = xyz$, $xy^0z \in L$, in questo caso questa condizione non è rispettata, $L \notin REG$.

PATTERN 1.2 (ROBA DA FARE CON I G-NFA)

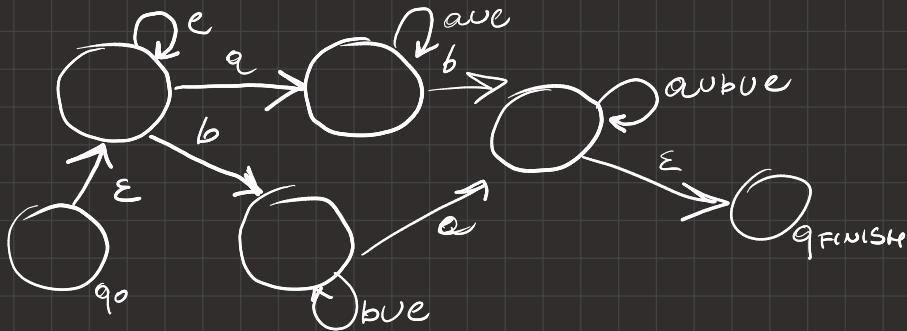
Problema 1.8

Sia $\Sigma = \{a, b, c\}$. Determinare un'espressione regolare $R \in \text{re}(\Sigma)$ descrivente il linguaggio di Σ composto dalle stringhe contenenti almeno una a ed almeno una b . Determinare inoltre un DFA D che riconosca lo stesso linguaggio.

PARTIAMO DAL DFA D



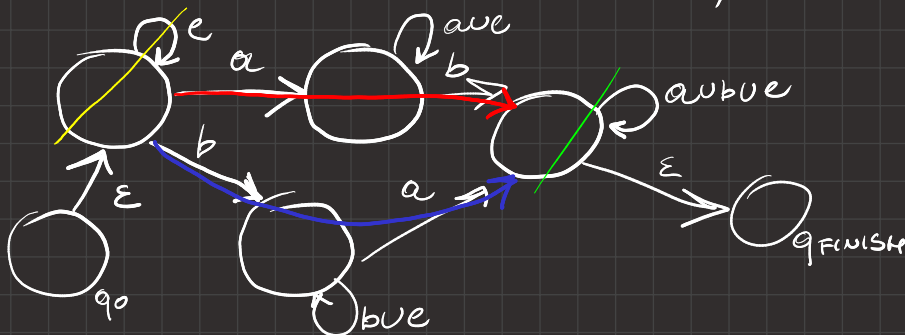
DA QUESTO COSTRUIAMO UN GNFA.



TRASALGO GLI ARCHI NULLI...

SAPPIAMO CHE $L(\text{GNFA}) \subseteq L(\text{REG})$, RIDUCENDO IL GNFA A 2 NODI OTTENIAMO UN SOLO ARCO CON ETICHETTA, QUELLA ETICHETTA E' LA ESPRESSIONE CHE CI INTERESSA.

RIDUCO QUESTO A 2 NODI,



$$E = e^* \left[\left[(b(bue)^* a) \cup (a(aue)^* b) \right] [aue]^* \right]$$

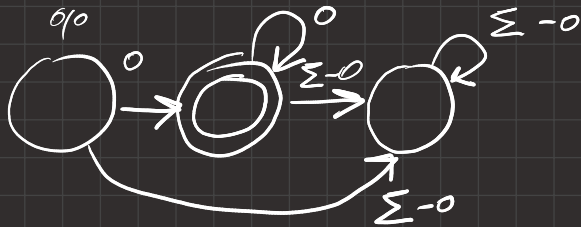
PATTERN 1.3 DIMOSTRARE CHE $L \in \text{REG}$

Problema 1.14

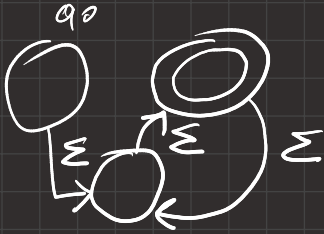
Dato il linguaggio $L = \{w \mid w \text{ termina con } 0 \text{ oppure ha lunghezza pari}\}$, dimostrare che $L \in \text{REG}$

BISOGNA DIMOSTRARE CHE $\exists$ DFA O NFA CHE LEGGE IL LINGUAGGIO L . INOLTRE SAPPIAMO CHE REG E' CHIUSO PER L'OPERAZIONE UNIONE.

D_1 : DFA CHE RICONOSCE $L_1 = \{w \mid w \text{ TERMINA CON } 0\}$



D_2 : DFA CHE RICONOSCE $L_2 = \{w \mid w \text{ HA LUNGHEZZA PARI}\}$



$L = L_1 \cup L_2$ E DATO CHE $L_1 \in REG, L_2 \in REG \Rightarrow L \in REG$.

PATTERN 1.4. BESTEMMIA MODULARE



Problema 1.10: Somma di cifre come multiplo di k

Sia L_k il linguaggio di tutte le stringhe su alfabeto $\Sigma = \{0, 1, \dots, 9\}$ la cui somma delle cifre è un multiplo di k , dove $k \in \mathbb{N}$. Descrivere formalmente un DFA che riconosca L_k per ogni $k \in \mathbb{N}$. Provare la correttezza del DFA proposto e disegnare il diagramma di transizione degli stati del DFA per il caso $k = 4$

PREMI VOI UNO COME CAZZO FA A FARE STO MOSTRO SE NON SE IMPARA UN PO' DI ESERCIZI A MEMORIA...

CREIAMO UN AUTOMA DFA $D = (Q, \Sigma, \sigma, q_0, F)$

DOVE

$$\sigma(q_i, a) = q_j \Leftrightarrow i + a = j \pmod{4}$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$F = \{q_0\}$$

$$q_0 = q_0$$

$$\Sigma = \{0, \dots, 9\}$$

DI MOSTRIAMO PER INDUZIONE SU i CHE È LA LUNGHEZZA DELLA STRINGA NUMERICA IN INPUT

E.B.
per $i=0 \vee i=1$ l'automa funziona
M.I.
supponiamo vero tutto questo fino a $i=n$ quindi che $\sigma^*(q_0, a_1 a_2 \dots a_n) = q_n \pmod{4}$

VERIFICHIAMO PER $i = n+1$

DOPO AVER LETTO $n+1$ CARATTERI SAREMO SU q_0, q_1, q_2, q_3 PER FORZA

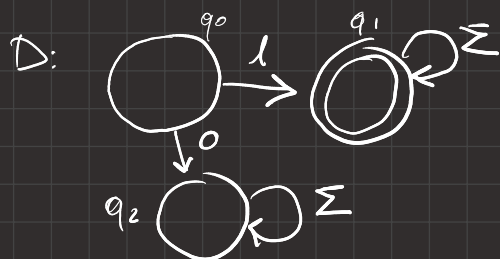
$$\sigma^*(q_m, a_1 a_2 a_3 \dots a_n a_{n+1}) = \sigma \left[\sigma^*(q_m, a_1 \dots a_n), a_{n+1} \right] = \sigma(q_m, a_{n+1}) = q_i \bmod(4)$$

VERO PER COSTRUZIONE

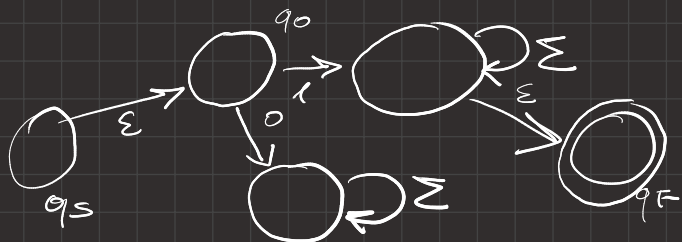
$$i \in L \iff \sigma^*(q_0, i) = q_0 \bmod(4) \iff L \in D.$$

2.1) DA ESP_REG $\rightarrow$ CFG

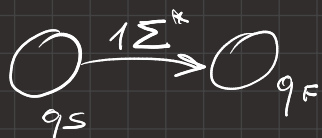
POSSIAMO CREARE UN DFA D , DIMOSTRARE CHE LEGGE L'ESPRESSIONE DATA TRASFORMANDO D IN UN GNFA E TRASFORMARE D IN UNA CFG DATO CHE $L(\text{DFA}) \subseteq \text{CFG}$



TRASFORMIAMOLO IN GNFA G



TRALASCIAMO GLI ARCHI CON ETICHETTA $= \emptyset$ E RIDUCIAMO G A 2 NODI



QUINDI $L(D) = L = L(R)$ DOVE R è l'espressione regolare data.

POSSO CREARE ORA $G \in \text{CFG}$ DA $D \in \text{PDA}$ PERCHÉ $L(\text{PDA}) \subseteq \text{CFL}$

$$G = (V = \{V_0, V_1, V_2\}, (\Sigma_G = 1, \Sigma), R = (*), S = V_0)$$

$$(*) \sigma(q_i, a) = q_j \iff V_i \rightarrow aV_j \quad \text{ALTE QUESTA REGOLA QUINDI } R =$$

$$V_0 \rightarrow 1V_1 \mid 0V_2$$

$$V_1 \rightarrow \epsilon \mid 0V_1 \mid 1V_1$$

$$V_2 \rightarrow 0V_2 \mid 1V_2$$

PATTERN 2.2.

Problema 2.2

Mostrare che la seguente grammatica è ambigua:

$$G: S \rightarrow TbT$$

$$T \rightarrow aTbT \mid bTaT \mid \varepsilon$$

Una grammatica si dice ambigua se esiste più di una derivazione sinistra per una stringa

$$S \Rightarrow TbT \Rightarrow bT \Rightarrow baTbT \Rightarrow babT \Rightarrow bab$$

oppure

$$S \Rightarrow TbT \Rightarrow bTaTbT \Rightarrow baTbT \Rightarrow babT \Rightarrow bab$$

SI, LA GRAMMATICA È AMBIGUA.

PATTERN 2.3

Problema 2.3

Data la seguente grammatica:

$$G: S \rightarrow WbT$$

$$T \rightarrow aWbT \mid bVaT \mid \varepsilon$$

$$W \rightarrow aWbW \mid \varepsilon$$

$$V \rightarrow bVaV \mid \varepsilon$$

Mostrare che $aabbbbaab \in L(G)$. Descrivere un PDA P tale che $L(G) = L(P)$

1)

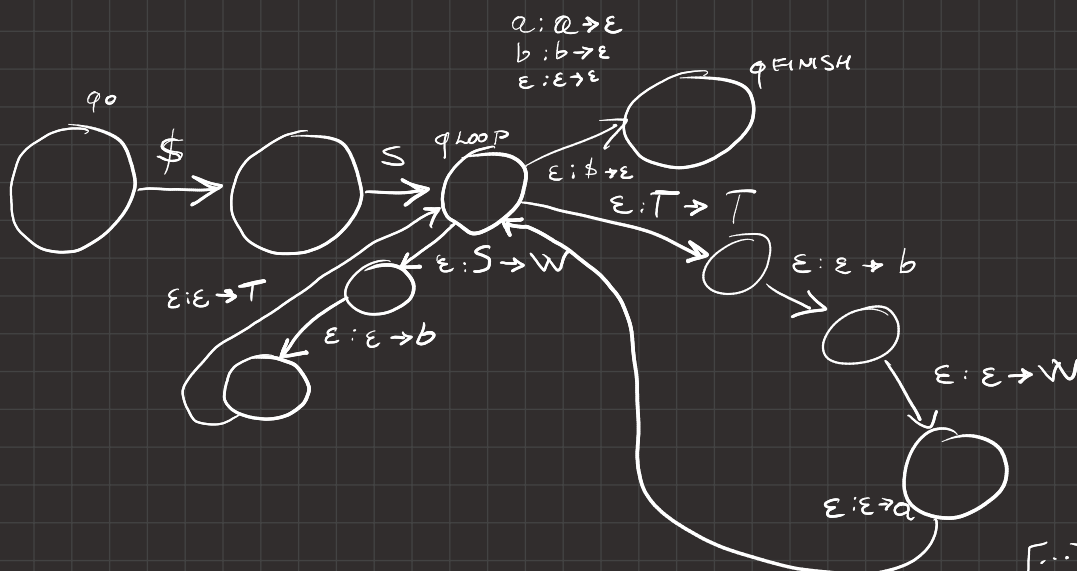
$$S \Rightarrow WbT \Rightarrow aWbTbT \Rightarrow aaWbTbTbT \Rightarrow aaWbTbTbbVaT$$

$$\Rightarrow aaWbTbTbbVa aWbT \Rightarrow aa bTbTbbVa aWbT \Rightarrow$$

$$aa bbbTbbVa aWbT \Rightarrow aa bbbbVa aWbT \Rightarrow aa bbbba aWbT \Rightarrow$$

$$aa bbbbaabT \Rightarrow aabbbbaab.$$

2) P:



[...] DOPO NE FACCIAMO UN ALTRO

PATTERN 2.4. PUMPING LEMMA PER LINGUAGGI ACONTESTUALI.

Problema 2.4

Classificare ognuno dei seguenti tre linguaggi come (a) regolare, (b) acontestuale ma non regolare, (c) non acontestuale:

1. $L_1 = \{w\#u \mid w, u \in \{0,1\}^* \text{ e } |w| = |u|\}$
2. $L_2 = \{w \in \{0,1\}^* \mid |w|_0 \text{ è dispari e } |w|_1 = 1\}$
3. $L_3 = \{w \in \{0,1,2\}^* \mid |w|_0 > |w|_1 \text{ e } |w|_0 > |w|_2\}$

$L_3 \notin CFL$ DIMOSTRIAMO

SUPPONIAMO SIA VERO ALLORA È VERO IL PUMPING LEMMA

CASO 1

$$w = xyzkv = 0^{p+1}1^p2^p$$

$$|y| \leq p, |y| > 0 \text{ e } |k| > 0$$

SE yzk CONTIENE SOLO "0" $w_2 = 0^{p+1-n}0^{n-k-l}1^p2^p$ per $k=0$

$$w_2 = 0^{p+1-k-l}1^p2^p$$

$$w_2 \in L_3$$

CASO 2

$$w = xyzkv = 0^{p+1}1^p2^p$$

$$w_3 = 0^{p+1}1^{p-n}(1^{n-k-l})^i1^k2^p$$

CASO 3

$$w = 012$$

SE CONTIENE PIU' ZEM CHE UNI PORTO $k=0$ ALTAMENTE k GIGANTE...

• FACCIAMO UN ESERCIZIO PER BENE CON STA ROBA

PATTERN 2.3

- Si consideri la seguente grammatica G:

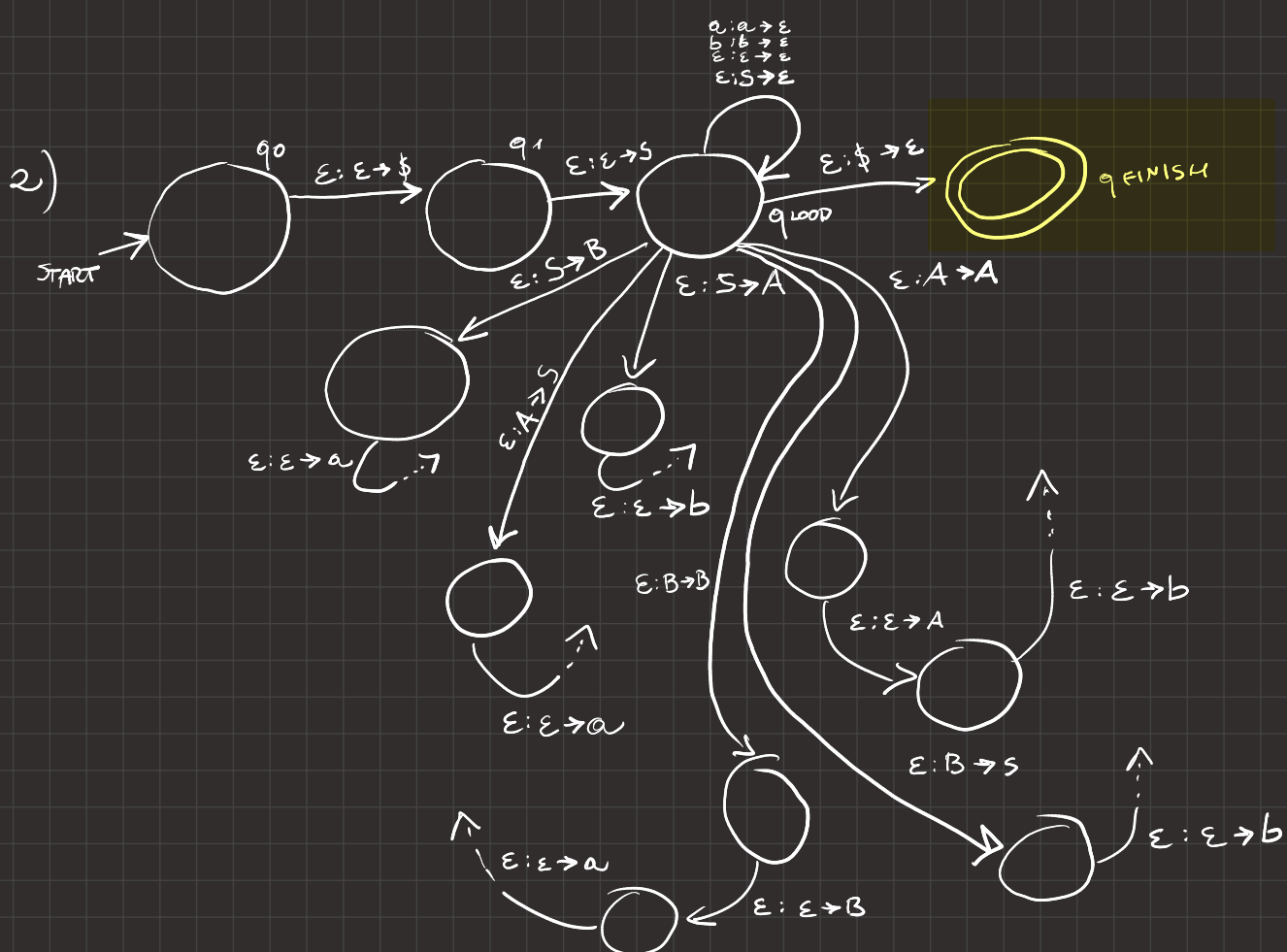
$$S \rightarrow aB \mid bA \mid \epsilon$$

$$A \rightarrow aS \mid bAA$$

$$B \rightarrow aBB \mid bS$$

Si fornisca una derivazione sinistra della stringa $bbaaab$. Si determini un PDA che riconosce lo stesso linguaggio generato da G.

$$1) S \Rightarrow bA \Rightarrow bbAA \Rightarrow bbaSA \Rightarrow bbaA \Rightarrow bbaaS \Rightarrow bbaaaB \Rightarrow bbaaabs \Rightarrow bbaaab$$



APPUNTO DI NOTAZIONE: $-->$ (RITORNO A q_{loop})

1 Automi

- Si classifichino i seguenti linguaggi come i) regolari, oppure ii) acontestuali ma non regolari, oppure iii) non acontestuali. In ciascun caso, si giustifichi la risposta.

(a) $\{w\#u : w, u \in \{0,1\}^* \text{ e } |w| = |u|\}$.

(b) $\left\{ w \in \{0,1\}^* : \begin{array}{l} w \text{ contiene un numero dispari di simboli '0'} \\ \text{ed un singolo simbolo '1'} \end{array} \right\}$.

(c) $\left\{ w \in \{0,1,2\}^* : \begin{array}{l} w \text{ contiene pi\u00f9 simboli '0' che simboli '1'} \\ \text{e pi\u00f9 simboli '0' che simboli '2'} \end{array} \right\}$.

- Si dimostri che un linguaggio \u00e8 regolare se e solo se esiste un NFA che lo riconosce.

$$(a) L_1 = \{w\#u : w, u \in \{0,1\}^* \text{ e } |w| = |u|\}$$

PROVIAMO CHE QUESTO LINGUAGGIO NON \u00c8 REGOLARE.

SE L_1 \u00c8 REGOLARE ALLORA VALE IL PUMPING LEMMA

PER I LINGUAGGI REGOLARI QUINDI $\forall x : |x| \geq p$ DOVE

p \u00c8 LA LUNGHEZZA DEL PUMPING $x = h y z$ DOVE

$|h y| \leq p$, $y > 0$ e $h y^k z \in L_1 \quad \forall k \geq 0$.

se prendo la stringa $x = 0^p \# 1^p$

$x = h y z = 0^{p-m} 0^m \# 1^p$, PONENDO $k=0$

$x = 0^{p-m} \# 1^p$ QUINDI $x \notin L_1$ VIOLANDO LA CONDIZIONE DEL PUMPING, QUINDI $L_1 \notin REG$.

FACILMENTE POSSIAMO COSTRUIRE $G \in CFG$

$S \rightarrow \#$

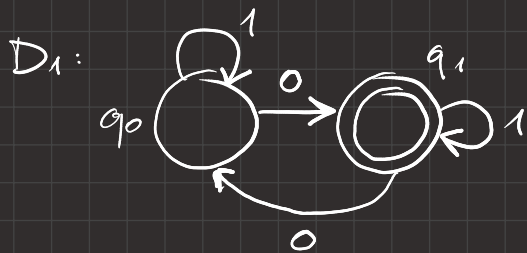
$\# \rightarrow 0 \# 1$

(b) $L_2 = \{ w \in \{0,1\}^* : w \text{ CONTIENE UN NUMERO DISPARI DI SIMBOLI } 0 \}$
ED UN SIMBOLO 1

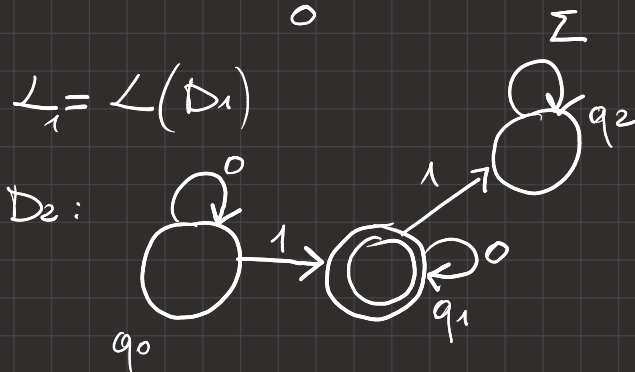
SE UN LINGUAGGIO È REGOLARE ALLORA È ANCHE ACONTESTUALE
DATO CHE $EFL \subseteq REG$

$L_1 = \{ w \in \{0,1\}^* : w \text{ CONTIENE UN NUMERO DISPARI DI SIMBOLI } 0 \}$

$L_2 = \{ w \in \{0,1\}^* : w \text{ CONTIENE UN SIMBOLO } 1 \}$



$L_1 = L(D_1)$



$L_2 = L(D_2)$

$L = L_1 \cup L_2$, DATO CHE $L_1 \in REG, L_2 \in REG$ ALLORA $L \in REG$

PERCHÉ I LINGUAGGI REGOLARI SONO CHIUSI PER L'OPERATORE
INTERSEZIONE

$$L_3 = \{ w \in \{0,1,2\}^* : w \text{ CONTIENE PIU' SIMBOLI 0 CHE 1} \}$$

SE L_3 E' ACONTESTUALE ALLORA DEVE RISPETTARE IL PUMPING

LEMMA PER I LINGUAGGI ACONTESTUALI QUINDI PRENDENDO $w \in L_3$

$$: |w| \geq p \text{ e } w = xyzh \text{ } |yzh| \leq p, |yh| > 0$$

$$w = xy^kzh^k \in L_3 \quad \forall k \geq 0 \wedge k \in \mathbb{N}$$

PRENDENDO LA STRINGA $0^{p+1}1^p2^p$

CASO 1 yzh TUTTI "0"

$$w = 0^{p+1-m-n-j} 0^j 0^m 0^n 1^p 2^p$$

$$y = 0^j$$

$$z = 0^m$$

$$h = 0^n$$

$$\text{con } k=0, w = 0^{p+1-m-n-j} (0^j)^0 (0^m)^0 (0^n)^0 1^p 2^p$$

$$w = 0^{p+1-j-m} 1^p 2^p \notin L_3$$

CASO 2 yzh TUTTI "1"

$$w = 0^{p+1} 1^{p-m-n-j} 1^m 1^n 1^j 2^p$$

$$y = 1^j$$

$$z = 1^m$$

$$h = 1^n$$

$$\text{con } k = \text{NUMERO ALTO}, w = 0^{p+1-m-n-j} (0^j)^{N.A.} (0^m)^{N.A.} (0^n)^{N.A.} 1^p 2^p$$

$$w = 0^{p+1} 1^{p+1+j} 2^p \notin L_3$$

CASO 3 yzh NON TUTTI "1"

DATO CHE $(yzh) \leq p$, yzh non può contenere "2"
SE yzh CADE TRA GLI "1" E GLI "0" dato che p è DISPARI
(ARBITRARIO) se ho più "1" che "0" in yzh PONGO $k =$
NUMERO GRANDE, SE ho più "0" che "1" in yzh pongo $k = 0$.
per costruzione $w \notin L_3$

in nessun modo $w \in L_3$ quindi $L_3 \notin CFL$.

Problema 3.5: $START-0_{TM}$ indecidibile

Dato il seguente linguaggio:

$$START-0_{TM} = \{ \langle M \rangle \mid M \text{ TM}, L(M) = \{0y \mid y \in \Sigma^*\} \}$$

dimostrare che è **indecidibile**, ossia che $START-0_{TM} \notin DEC$

$$START-0_{TM} = \{ \langle M \rangle \mid M \text{ TM}, L(M) = \{0y \mid y \in \Sigma^*\} \}$$

SE $START-0_{TM}$ È DECIDIBILE ALLORA D È IL SUO DECISORE

CREIAMO ORA R CHE PRENDE IN INPUT $\langle M, w \rangle$

LANCIA D SU INPUT $\langle M \rangle$

SE D ACCETTA, IL LINGUAGGIO DI M È DELLA FORMA $\{0y \mid y \in \Sigma^*\}$
CONTROLLO CHE $w \in \{0y \mid y \in \Sigma^*\}$ SE SÌ

R ACCETTA, ALTRIMENTI R RIFIUTA

SE D RIFIUTA, IL LINGUAGGIO DI M NON È DI QUELLA
FORMA, R RIFIUTA.

abbiamo creato un DECISORE PER ATM (ma) DATO CHE
 ATM È INDECIDIBILE R NON ESISTE E NEMMENO D.



Ciao! Quelli che hai postato sono classici esempi di **riduzione polinomiale** (o riduzione per mapping $A \leq_m B$). Lo schema per risolvere questi problemi è quasi sempre lo stesso: si "nasconde" un problema difficile (di solito A_{TM} , il problema dell'accettazione) dentro la definizione di una nuova macchina di Turing.



Ecco uno schema universale in 5 passi per dimostrare che un linguaggio L è indecidibile.

Schema Universale di Riduzione ($A_{TM} \leq_m L$)

1. Definizione della Riduzione

Dichiara l'obiettivo: "Dimostriamo che $A_{TM} \leq_m L$ tramite una funzione calcolabile f che trasforma un'istanza $\langle M, w \rangle$ in un'istanza $\langle M' \rangle$ (o quello che richiede il linguaggio target)."

2. Costruzione della Macchina M' (Il "Guscio")

Descrivi come costruire una nuova macchina M' che riceve un input x . La struttura interna di M' deve essere:

1. **Filtro iniziale:** Controlla se x soddisfa la condizione del linguaggio target L .
 - Esempio (3.5): Se x non inizia con $\emptyset$, rifiuta.
 - Esempio (3.8): Se $|x|$ è pari, rifiuta.
2. **Simulazione "Cuore":** Se x ha superato il filtro, esegui M su w (l'input originale del problema A_{TM}).
3. **Output:** Se $M(w)$ accetta, allora M' accetta x .

3. Analisi del Comportamento (I due casi)

Devi dimostrare che la riduzione "funziona" in entrambi i sensi:

- **Caso SI** ($\langle M, w \rangle \in A_{TM}$): Se M accetta w , allora M' accetterà **tutte e sole** le stringhe che hanno superato il filtro al punto 2. Di conseguenza, $L(M')$ sarà esattamente l'insieme richiesto dal linguaggio L . Quindi $\langle M' \rangle \in L$.
- **Caso NO** ($\langle M, w \rangle \notin A_{TM}$): Se M non accetta w (loop o rifiuto), allora M' non accetterà **mai** nulla, indipendentemente dal filtro. Di conseguenza, $L(M') = \emptyset$. Poiché l'insieme vuoto di solito non soddisfa le condizioni di L , allora $\langle M' \rangle \notin L$.

4. Conclusione Logica

Afferma che:

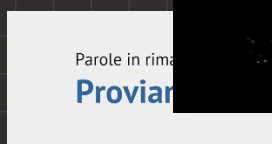
$$\langle M, w \rangle \in A_{TM} \iff f(\langle M, w \rangle) \in L$$

Questo prova che f è una riduzione valida.

5. Verdetto Finale

Poiché A_{TM} è noto per essere indecidibile ($A_{TM} \notin DEC$), allora anche L deve essere indecidibile. Se L fosse decidibile, potremmo usare il decisore di L per decidere A_{TM} , il che è un assurdo.

PROVIAMO!



PATTERN:

- 1
- 1.1 DIMOSTRA CHE UN LINGUAGGIO NON E' REGOLARE.
 - 1.2 PASSAGGI TRA REG, LGNFA, LNFA, DFA, LRE).
 - 1.3 DIMOSTRA CHE UN LINGUAGGIO E' REGOLARE.
 - 1.4 BESTEMMIA MODULARE.

- 2(1)
- 2.1 DA ESP-REG $\rightarrow$ CFG
 - 2.2 AMBIGUITA'
 - 2.3 DA GRAMMATICA $\rightarrow$ PDA
 - 2.4 DIMOSTRA CHE UN LINGUAGGIO E' ACONTESTUALE.

- 3
- 3.1 DIMOSTRA IRRI CONOSCIBILITA'.
 - 3.2 DIMOSTRA INDECIDIBILITA'.
 - 3.3 ALGORITHM, DIMOSTRA DECIDIBILITA'.